

CHAPTER 1 · SUB-CHAPTER 1.7

Generics

Generic Classes & Interfaces · Bounded Type Parameters · Wildcards & PECS
Type Erasure · Generic Methods · Bridge Methods · Generics in Practice

~20 Pages

52+ Q&A

Adv. Questions

Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — Generic Classes & Interfaces

1.1 Declaring Generic Classes

```
// Generic class with single type parameter
class Box<T> {
    private T value;
    public Box(T value) { this.value = value; }
    public T getValue() { return value; }
    public void setValue(T value) { this.value = value; }
    @Override public String toString() { return "Box[" + value + "]"; }
}

// Generic class with multiple type parameters
class Pair<A, B> {
    private final A first;
    private final B second;
    public Pair(A first, B second) { this.first = first; this.second = second; }
    public A getFirst() { return first; }
    public B getSecond() { return second; }
    // Static factory method
    public static <A, B> Pair<A, B> of(A a, B b) { return new Pair<>(a, b); }
}

// Usage
Box<String> strBox = new Box<>("Hello"); // explicit type
Box<Integer> intBox = new Box<>(42); // autoboxing
Box<?> unknown = new Box<>("wildcard"); // unknown type
Pair<String, Integer> p = Pair.of("age", 30);

// Raw types (pre-Java-5 / legacy – avoid!)
Box rawBox = new Box("raw"); // unchecked warning – erased to Object
Object val = rawBox.getValue(); // compiles but loses type safety
```

- Type parameter naming conventions: T=Type, E=Element, K=Key, V=Value, N=Number, R=Return type, S/U/V=2nd/3rd/4th types
- Diamond operator <> (Java 7+): compiler infers type from context — new HashMap<>() becomes new HashMap<>()
- Raw types bypass all compile-time generic checks; mixing raw and generic types causes heap pollution

1.2 Generic Interfaces

```
// Generic interface
interface Repository<T, ID> {
    T findById(ID id);
    List<T> findAll();
    T save(T entity);
    void delete(ID id);
}

// Implementing with specific type
class UserRepository implements Repository<User, Long> {
    @Override public User findById(Long id) { /* ... */ return null; }
    @Override public List<User> findAll() { return new ArrayList<>(); }
    @Override public User save(User u) { return u; }
    @Override public void delete(Long id) { /* ... */ }
}

// Implementing keeping type parameter
class InMemoryRepository<T> implements Repository<T, Integer> {
    private final Map<Integer, T> store = new HashMap<>();
    @Override public T findById(Integer id) { return store.get(id); }
    @Override public List<T> findAll() { return new ArrayList<>(store.values()); }
    @Override public T save(T entity) { store.put(store.size()+1, entity); return entity; }
    @Override public void delete(Integer id){ store.remove(id); }
}

// Generic interface with default method (Java 8+)
interface Transformer<I, O> {
    O transform(I input);
    default <R> Transformer<I, R> andThen(Transformer<O, R> after) {
```

```
return input -> after.transform(this.transform(input));  
}  
}
```

Section 2 — Bounded Type Parameters

2.1 Upper Bounded Type Parameters

```
// T must extend (implement) Comparable<T>
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}

// T must extend Number – can call Number methods on T
public static <T extends Number> double sum(List<T> list) {
    return list.stream().mapToDouble(Number::doubleValue).sum();
}

// Multiple bounds: T extends class AND implements interfaces
// Class bound must come FIRST, interface bounds after &
public static <T extends Comparable<T> & Serializable & Cloneable>
void processComparableSerializable(T value) {
    System.out.println(value.compareTo(value)); // 0
}

// Bounded in class definition
class SortedList<T extends Comparable<T>> {
    private final List<T> list = new ArrayList<>();
    public void add(T item) {
        list.add(item);
    }
    Collections.sort(list); // valid because T is Comparable
}

public T min() { return list.isEmpty() ? null : list.get(0); }
}
```

2.2 Recursive Bounds & F-Bounded Quantification

```
// F-bounded: T refers to itself in its own bound
// Enables type-safe compareTo in subclasses
class Animal<T extends Animal<T>> implements Comparable<T> {
    @Override public int compareTo(T other) { return 0; }
}

class Dog extends Animal<Dog> {} // T = Dog; compareTo(Dog) – type-safe

// The standard Enum pattern uses this:
// class Enum<E extends Enum<E>> – every enum implicitly has this bound
// This enables: E.compareTo(E) being type-safe (no Object cast)

// Builder pattern with self-referential generic
abstract class Builder<T, B extends Builder<T, B>> {
    @SuppressWarnings("unchecked")
    protected B self() { return (B) this; }
    public B withName(String name) { /* set name */ return self(); }
    public abstract T build();
}

class PersonBuilder extends Builder<Person, PersonBuilder> {
    @Override public Person build() { return new Person(); }
}
```

Section 3 — Wildcards & PECS

PECS: Producer Extends, Consumer Super. If the collection PRODUCES values you READ from it, use `.` If it CONSUMES values you WRITE to it, use `?`. If both, use the exact type `.`

3.1 Unbounded Wildcard <?>

```
// <?> means: I don't care about the element type – I only use Object API
void printAll(List<?> list) {
    for (Object item : list) { // only Object methods available
        System.out.println(item);
    }
}

// list.add("x"); // COMPILE ERROR – can't add (type unknown)
// list.add(null); // OK – null is always valid

// Different from List<Object>:
List<String> strings = List.of("a", "b");
List<Object> objects = new ArrayList<>();
// objects = strings; // COMPILE ERROR – List<String> is NOT List<Object>
List<?> wildcard = strings; // OK – List<?> accepts any List<T>
```

3.2 Upper Bounded Wildcard <? extends T> — Producer Extends

```
// <? extends Number>: list holds some subtype of Number (Integer, Double, etc.)
double sumNumbers(List<? extends Number> src) {
    double total = 0;
    for (Number n : src) total += n.doubleValue(); // reading as Number: OK
    return total;
}

sumNumbers(new ArrayList<Integer>()); // OK
sumNumbers(new ArrayList<Double>()); // OK

// Cannot add to <? extends Number> – compiler doesn't know exact type
List<? extends Number> nums = new ArrayList<Integer>();
// nums.add(3.14); // COMPILE ERROR – might be List<Integer>!
// nums.add(1); // COMPILE ERROR
nums.add(null); // OK

// Real-world: Collections.copy uses PECS
// static <T> void copy(List<? super T> dest, List<? extends T> src)
List<Integer> src = List.of(1, 2, 3);
List<Number> dest = new ArrayList<>(List.of(0, 0, 0));
Collections.copy(dest, src); // dest consumes (super), src produces (extends)
```

3.3 Lower Bounded Wildcard <? super T> — Consumer Super

```
// <? super Integer>: list holds Integer or any supertype (Number, Object)
void addIntegers(List<? super Integer> dest, int count) {
    for (int i = 0; i < count; i++) dest.add(i); // writing Integer: OK
}

addIntegers(new ArrayList<Integer>(), 5); // OK
addIntegers(new ArrayList<Number>(), 5); // OK
addIntegers(new ArrayList<Object>(), 5); // OK

// Cannot read typed values from <? super Integer>
List<? super Integer> dest2 = new ArrayList<Number>();
// Integer x = dest2.get(0); // COMPILE ERROR – might be Number or Object
Object o = dest2.get(0); // only Object available when reading

// Summary table:
// <? extends T>: get() returns T | add() BLOCKED (except null)
// <? super T> : get() returns Object | add(T) allowed
// <T> : get() returns T | add(T) allowed – most flexible
```

Wildcard	Can Read As	Can Write	Use Case
	Object	null only	Print/count elements, no type-specific ops

	T (upper bound)	null only	Reading/processing; Producer pattern
	Object	T and subtypes	Writing/collecting; Consumer pattern
(exact)	T	T	Both reading and writing needed

Section 4 — Type Erasure

Type erasure: Java generics exist at compile time only. The compiler enforces type rules then erases all generic information. At runtime, `List` and `List` are the same class: `java.util.List`. This was done for backward compatibility with pre-Java-5 bytecode.

4.1 What Erasure Does

```
// Source code (before erasure)
class Container<T extends Comparable<T>> {
    private T value;
    public T get() { return value; }
    public void set(T v) { this.value = v; }
    public boolean bigger(T other) { return value.compareTo(other) > 0; }
}

// After erasure (what JVM sees – approximately)
class Container { // T erased
    private Comparable value; // T -> first bound (Comparable)
    public Comparable get() { return value; }
    public void set(Comparable v) { this.value = v; }
    public boolean bigger(Comparable other) { return value.compareTo(other) > 0; }
}

// Unbounded T erases to Object; bounded T erases to its first bound
// Consequences of type erasure:
// 1. Cannot use instanceof with parameterized type:
// if (obj instanceof List<String>) {} // COMPILE ERROR
// if (obj instanceof List<?>) {} // OK (unbounded wildcard)
// 2. Cannot create instance of type parameter:
// T t = new T(); // COMPILE ERROR
// 3. Cannot create generic array:
// T[] arr = new T[10]; // COMPILE ERROR (heap pollution)
// 4. Cannot use in catch clause:
// catch (SomeException<T> e) {} // COMPILE ERROR
// 5. Cannot overload by generic type alone:
// void process(List<String> list) {}
// void process(List<Integer> list) {} // COMPILE ERROR – same erasure
// 6. Static members cannot use class type parameter:
// static T instance; // COMPILE ERROR
```

4.2 Heap Pollution & Unchecked Warnings

```
// Heap pollution: a variable of parameterized type refers to wrong type
List<String> strings = new ArrayList<>();
List rawList = strings; // raw type assignment – unchecked
rawList.add(42); // adds Integer to List<String> – NO compile error!
String s = strings.get(0); // ClassCastException at RUNTIME

// @SuppressWarnings("unchecked") – use sparingly, only when you KNOW it's safe
@SuppressWarnings("unchecked")
static <T> T cast(Object obj) {
    return (T) obj; // unchecked cast – OK when caller guarantees type
}

// Reifiable types vs non-reifiable
// Reifiable: type fully available at runtime – String[], Object[], int[], List<?>
// Non-reifiable: List<String>, Map<K,V> – type info erased
// Only reifiable types can be used in instanceof, new[], catch
// Checking parameterized type at runtime – use Class<T> token
```

```
class TypedList<T> {  
    private final Class<T> type;  
    private final List<T> list = new ArrayList<>();  
    public TypedList(Class<T> type) { this.type = type; }  
    public void add(Object o) {  
        if (!type.isInstance(o))  
            throw new ClassCastException("Expected " + type.getName());  
        list.add(type.cast(o)); // safe cast  
    }  
}
```


Section 5 — Generic Methods

```
// Generic method – type parameter declared before return type
public static <T> T identity(T value) { return value; }
public static <T> List<T> listOf(T a, T b) { return List.of(a, b); }

// Type inference at call site
String s = identity("hello"); // T inferred as String
Integer i = identity(42); // T inferred as Integer

// Explicit type witness when inference fails
List<String> empty = Collections.<String>emptyList();
// Usually not needed with Java 8+ improved target-type inference

// Generic method in non-generic class
class Utils {
    // Swap two elements in a list
    public static <T> void swap(List<T> list, int i, int j) {
        T temp = list.get(i);
        list.set(i, list.get(j));
        list.set(j, temp);
    }

    // Generic filter
    public static <T> List<T> filter(List<T> list, Predicate<T> pred) {
        return list.stream().filter(pred).collect(Collectors.toList());
    }

    // Generic zip: combine two lists element-by-element
    public static <A, B, R> List<R> zip(List<A> as, List<B> bs,
        BiFunction<A, B, R> combiner) {
        List<R> result = new ArrayList<>();
        int size = Math.min(as.size(), bs.size());
        for (int k = 0; k < size; k++) result.add(combiner.apply(as.get(k), bs.get(k)));
        return result;
    }

    // Generic max with bounded type
    public static <T extends Comparable<T>> Optional<T> max(Collection<T> col) {
        return col.stream().max(Comparator.naturalOrder());
    }

    // Generic deep copy using serialization (requires Serializable bound)
    @SuppressWarnings("unchecked")
    public static <T extends Serializable> T deepCopy(T obj) throws Exception {
        ByteArrayOutputStream bos = new ByteArrayOutputStream();
        new ObjectOutputStream(bos).writeObject(obj);
        return (T) new ObjectInputStream(
            new ByteArrayInputStream(bos.toByteArray())).readObject();
    }
}
```

Section 6 — Bridge Methods & Covariant Return Types

```
// Bridge method – compiler generates for erasure + polymorphism
interface Processor<T> {
    T process(T input);
}

class StringProcessor implements Processor<String> {
    @Override
    public String process(String input) { return input.toUpperCase(); }
    // Compiler ALSO generates (invisible to you, visible via javap):
    // public synthetic bridge Object process(Object input) {
```

```
// return process((String) input); // delegates with cast
// }
}
// Why? The JVM needs: Processor.process(Object)Object for polymorphism.
// Without bridge, StringProcessor wouldn't override Processor.process(Object)Object.
// View bridge methods with: javap -p -verbose StringProcessor
// Flags: ACC_SYNTHETIC, ACC_BRIDGE on the generated method
// Covariant return type also generates bridge:
class Animal {
    public Animal clone() { return new Animal(); }
}
class Dog extends Animal {
    @Override
    public Dog clone() { return new Dog(); } // covariant return
    // Bridge: public Animal clone() { return clone(); } -- generated
}
// Impact on reflection:
for (Method m : StringProcessor.class.getDeclaredMethods()) {
    System.out.println(m.getName() + " bridge=" + m.isBridge()
        + " synthetic=" + m.isSynthetic());
}
// process bridge=false synthetic=false <- your method
// process bridge=true synthetic=true <- compiler-generated bridge
```

Section 7 — Generics in Practice

7.1 Generic Result Type (Success / Failure)

```
// Type-safe Result monad – avoids checked exceptions for expected failures
sealed interface Result<T> permits Result.Success, Result.Failure {
    record Success<T>(T value) implements Result<T> {}
    record Failure<T>(String message, Throwable cause) implements Result<T> {}
    static <T> Result<T> success(T value) { return new Success<>(value); }
    static <T> Result<T> failure(String msg, Throwable t) {
        return new Failure<>(msg, t);
    }
}

default <R> Result<R> map(Function<T, R> fn) {
    return switch (this) {
        case Success<T> s -> Result.success(fn.apply(s.value()));
        case Failure<T> f -> Result.failure(f.message(), f.cause());
    };
}

default T getOrElse(T defaultValue) {
    return switch (this) {
        case Success<T> s -> s.value();
        case Failure<T> f -> defaultValue;
    };
}

// Usage
Result<User> result = userService.findUser(id);
String name = result.map(User::getName).getOrElse("Unknown");
```

7.2 Type Token Pattern (TypeSafe Heterogeneous Container)

```
// Problem: Map<Class<T>?, Object> loses type safety
// Solution: type token – Class<T> carries runtime type info despite erasure

class TypeSafeContainer {
    private final Map<Class<T>?, Object> map = new HashMap<>();
    public <T> void put(Class<T> type, T value) {
        map.put(Objects.requireNonNull(type), Objects.requireNonNull(value));
    }
    public <T> T get(Class<T> type) {
        return type.cast(map.get(type)); // safe cast; throws if wrong type
    }
}

TypeSafeContainer c = new TypeSafeContainer();
c.put(String.class, "hello");
c.put(Integer.class, 42);
String s = c.get(String.class); // "hello" – type safe
Integer i = c.get(Integer.class); // 42 – type safe

// Limitation: can't store List<String> vs List<Integer> – same Class token
// Solution: Guava TypeToken / Jackson TypeReference for parameterized types
// TypeReference<List<String>> ref = new TypeReference<List<String>>(){};
// This captures ParameterizedType at compile time via anonymous subclass trick
```

7.3 Generic Builder Pattern

```
// Fluent generic builder with type-safe inheritance
class HttpRequest {
```

```

final String method, url;
final Map<String, String> headers;
protected HttpRequest(Builder<?> b) {
    this.method = b.method;
    this.url = b.url;
    this.headers = Map.copyOf(b.headers);
}

static class Builder<B> extends Builder<B>&gt; {
    String method = "GET", url;
    Map<String, String> headers = new LinkedHashMap<>();
    @SuppressWarnings("unchecked")
    public B method(String m) { this.method = m; return (B) this; }
    @SuppressWarnings("unchecked")
    public B url(String u) { this.url = u; return (B) this; }
    @SuppressWarnings("unchecked")
    public B header(String k, String v) { headers.put(k,v); return (B) this; }
    public HttpRequest build() { return new HttpRequest(this); }
}

// Subclass extends builder without casting
class AuthenticatedRequest extends HttpRequest {
    final String token;
    protected AuthenticatedRequest(Builder b) {
        super(b); this.token = b.token;
    }
    static class Builder extends HttpRequest.Builder<Builder> {
        String token;
        public Builder token(String t) { this.token = t; return this; }
        @Override public AuthenticatedRequest build() {
            return new AuthenticatedRequest(this);
        }
    }

    // Fluent chaining returns correct subtype – no cast needed:
    AuthenticatedRequest req = new AuthenticatedRequest.Builder()
        .method("POST").url("/api/data").token("Bearer xyz").build();

```

Section 8 — Common Pitfalls & Anti-Patterns

Anti-Pattern	Why It Fails	Correct Approach
<code>new T()</code>	Type erasure: <code>T</code> is <code>Object</code> at runtime; no-arg constructor not guaranteed	Pass <code>Class</code> token and use <code>Class.newInstance()</code> or <code>Supplier</code>
<code>T[] arr = new T[n]</code>	Generic array creation — heap pollution risk	Use <code>(T[]) new Object[n]</code> with <code>@SuppressWarnings</code> , or <code>List</code>
<code>catch (GenException e)</code>	Catch clause requires reifiable type; <code>T</code> is erased	Catch raw <code>GenException</code> and check type manually
<code>instanceof List</code>	Parameterized types not reifiable; always use <code>instanceof List</code>	Use <code>instanceof List</code> then cast with runtime check
Static <code>T</code> field	<code>T</code> is per-instance; static fields are per-class	Use static generic method or non-generic static field
Overload by generic type	Erasure makes <code>List</code> and <code>List</code> same signature	Use different method names or differently named overloads
Raw type usage	Bypasses all type checking; heap pollution risk	Always specify type parameter; use for unknown type
Ignore unchecked warnings	May indicate real heap pollution bug	Fix the code or verify safety and <code>@SuppressWarnings</code> with comment

```
// Correct generic array workaround patterns
// Option 1: Unchecked cast with suppression (common in JDK itself)
@SuppressWarnings("unchecked")
T[] createArray(int size) {
    return (T[]) new Object[size]; // safe if T is not known to caller
}

// Option 2: Use Class<T> token for type-safe creation
T[] createArray(Class<T> type, int size) {
    return (T[]) java.lang.reflect.Array.newInstance(type, size);
}

// Option 3: Use List<T> instead of T[] – usually the right answer
List<T> createList(int size) {
    return new ArrayList<>(size);
}

// Generic stack using List internally (avoids array issues)
class GenericStack<E> {
    private final List<E> elements = new ArrayList<>();
    public void push(E e) { elements.add(e); }
    public E pop() { return elements.remove(elements.size()-1); }
    public E peek() { return elements.get(elements.size()-1); }
    public boolean isEmpty() { return elements.isEmpty(); }
    public int size() { return elements.size(); }
}
```

Interview Q&A; — 52 Questions

Purple [ADV] questions target senior/principal-level interviews. Standard questions suit mid-level Java engineers.

Part 1 — Generic Classes, Interfaces & Methods

Q1. What is the purpose of generics in Java?

Generics provide compile-time type safety, eliminate explicit casts, and enable code reuse across different types. Instead of List of Object requiring casts, List guarantees all elements are Strings, with ClassCastException prevented at compile time rather than runtime. They also communicate intent: Map> is self-documenting.

Q2. What is a raw type and why should you avoid it?

A raw type is a generic class/interface used without type arguments: List instead of List. It defeats all compile-time type checking — the compiler treats it like a pre-Java-5 class. You lose type safety silently: a raw List accepts any object; retrieving causes ClassCastException at an unexpected place. Only use raw types when interfacing with legacy pre-generics APIs, and isolate that code as much as possible.

Q3. What is the diamond operator <> and what was coding like before it?

Diamond operator (Java 7+) lets the compiler infer type parameters on the right side of an assignment: Map> map = new HashMap<>(); Before Java 7 you had to repeat the full type: new HashMap>(). The diamond is just syntactic sugar — it improves readability for complex nested generics without losing type safety.

Q4. What are type parameter naming conventions?

Standard conventions: T=general Type, E=Element (collections), K=Key, V=Value (maps), N=Number, R=Return type, S/U/V=2nd/3rd/4th types. These are conventions, not rules — you can use any valid identifier. For domain-specific generics, descriptive names like are clearer than . Single uppercase letters are traditional for readability — they stand out from class names.

Q5. Can a generic class extend a non-generic class? Can a non-generic class extend a generic class?

Yes to both. Generic extends non-generic: class NumberBox extends Object {}. Non-generic extends generic: class StringList extends ArrayList {} — the non-generic subclass fixes the type argument. This is common for domain-specific list types. Also: class RawList extends ArrayList {} — extends with raw type (not recommended).

Q6. What is the difference between a bounded type parameter and a bounded wildcard?

Bounded type parameter is used in class/method declarations — T becomes a placeholder usable throughout the declaration. Bounded wildcard is used in variable/parameter types — it means 'some specific unknown subtype'. Key difference: with you can relate multiple uses of T to the same type. With each occurrence is independent. Example: void copy(List src, List dest) — src and dest have the SAME type. void copy(List src, List dest) — they could be different types.

Q7. How do you call a generic static method explicitly specifying the type?

Use the type witness syntax: Collections.emptyList(). Before the method name, specify the type in angle brackets. Usually unnecessary with Java 8+ improved target-type inference, but needed when inference fails: when the return type is used in an overloaded call or assigned to Object. Example: Object o = Collections.emptyList(); vs Object o = Collections.emptyList(); (both compile, but the latter gives a raw type warning).

Q8. Can you overload a method that differs only in generic type argument?

No. After erasure, List and List become the same type List. Two methods void process(List list) and void process(List list) have identical erasures — compile error: 'erasure of process(List) is the same'. Solution: use different method names (processStrings, processIntegers) or use a single method with an additional Class parameter.

Part 2 — Wildcards & PECS

Q9. Explain PECS with a concrete example.

Producer Extends, Consumer Super. `Collections.copy(List dest, List src)`: `src` PRODUCES elements (you read `T` from it) so it uses `extends`. `dest` CONSUMES elements (you write `T` to it) so it uses `super`. Another example: `void pushAll(Iterable src, Stack stack)` — you read from `src` (producer=`extends`). `void popAll(Collection dst, Stack stack)` — you write to `dst` (consumer=`super`). Use exact type when you need both read and write.

Q10. What is the difference between List and List?

`List` only accepts `List` itself — not `List` or `List`, because generics are invariant. You can add any `Object` to `List`. `List` accepts ANY `List` for any `T` (`List`, `List`, etc.). You cannot add anything to `List` (except `null`) because the compiler doesn't know the actual type. `List` is useful for methods that only need to read elements as `Object` or just operate on the list structure (`size`, `clear`, etc.).

Q11. Why can't you add to a List?

The compiler doesn't know the exact type. `List` could be a `List`, `List`, or `List`. If you could add a `Double` to what's actually a `List`, you'd corrupt the list. The compiler prevents this conservatively — no add is allowed (except `null`). This is the 'covariance' safety rule: you can read safely (always a `Number`), but you can't write safely.

Q12. Why can't you read typed values from a List?

`List` could be `List`, `List`, or `List`. The compiler doesn't know which, so `get()` can only return `Object`. You know you can write `Integer` (and subtypes), because whatever the list holds, an `Integer` IS-A supertype-of-`Integer`. But reading back, you'd need a cast: `Object o = list.get(0); Integer i = (Integer) o;` — risky if list holds `Numbers`.

Q13. When would you use an unbounded wildcard ?

When you only use `Object` API on the elements, or operate on the collection structurally. Examples: printing all elements (`System.out.println` accepts `Object`), counting elements (`list.size()`), checking `isEmpty()`, or passing to a method that accepts `List` for maximum flexibility. Also: static utility methods that don't care about element type, like `void printList(List list)` or `int count(Collection col)`.

Q14. What does wildcard capture mean and what is the capture helper pattern?

Wildcard capture: when a method parameter is `?`, the compiler assigns an anonymous type for that specific invocation — 'capturing' the wildcard. The helper pattern: when you want to perform operations requiring the type to be consistent across multiple uses, use a generic helper method. Example: `void swap(List list, int i, int j)` calls helper: `void swapHelper(List list, int i, int j) { T t=list.get(i); list.set(i,list.get(j)); list.set(j,t); }` The helper captures the wildcard type as `T`, enabling type-safe operations.

Part 3 — Type Erasure**Q15. What is type erasure and why does Java use it?**

Generic type information (type arguments like `<T>`, `<E>`) is erased by the compiler and replaced with bounds (or `Object`) at runtime. `List` and `List` are both just `List` at runtime. Java uses erasure for backward compatibility — pre-Java-5 bytecode (compiled without generics) could still use Java 5+ generic classes without recompilation. This was a deliberate trade-off: generics as a compile-time-only feature.

Q16. What are the practical limitations caused by type erasure?

1) Cannot do `instanceof List` — use `instanceof List`. 2) Cannot create `new T()` — pass `Class` and use reflection or `Supplier`. 3) Cannot create `T[]` — use `(T[])new Object[n]` or `Array.newInstance`. 4) Cannot catch generic exception `catch(GenEx e)`. 5) Cannot overload by generic type alone. 6) Cannot use class `T` in a static context (`T.class` is illegal). 7) Reflection: `getGenericType()` returns `ParameterizedType` but only for fields/methods, not for local variables.

Q17. What is heap pollution and when does it occur?

Heap pollution: a variable of parameterized type refers to an object that is not of that parameterized type. Occurs when mixing raw types and generic types, or when using unchecked casts. Example: `List strings = new ArrayList<>(); ((List)strings).add(42);` The list is 'polluted' — it contains an `Integer` masquerading as `String`. The `ClassCastException` will occur only when you try to read the element as `String`, potentially far from the actual bug.

Q18. What is `@SuppressWarnings("unchecked")` and when is it acceptable to use?

`@SuppressWarnings("unchecked")` suppresses compiler warnings about unchecked casts and raw type operations. Acceptable when: (1) You have verified by reasoning that the cast is safe (e.g., you control all insertions). (2) The warning is in a utility method that is inherently unchecked by design (like a type-safe container using `Object` internally). Always accompany with a comment explaining WHY it's safe. Never use it to silence warnings you don't understand — those are real type safety holes.

Q19. How can you get generic type information at runtime despite erasure?

For fields and method return/parameter types, reflection preserves generic signatures in bytecode metadata: `field.getGenericType()` returns `ParameterizedType` for `List`. The super-type token pattern exploits this: `new TypeReference<List<String>>(){} creates an anonymous subclass; getClass().getGenericSuperclass() returns the ParameterizedType with actual type arguments. Jackson, Gson, and Guava TypeToken all use this technique for JSON deserialization into generic types like List<User>.`

Part 4 — Bridge Methods & Covariance**Q20. What is a bridge method and when does the compiler generate one?**

A synthetic method generated by the compiler to preserve polymorphism after type erasure. When a class implements a generic interface: class `SP` implements `Processor`, the compiler generates `process(Object)` that delegates to `process(String)`. The JVM needs the erased signature (`process(Object)`) for virtual dispatch. Bridge methods are also generated for covariant return types. View them with: `javap -p -verbose ClassName` — they are marked `ACC_SYNTHETIC + ACC_BRIDGE`.

Q21. How do bridge methods interact with reflection?

`getDeclaredMethods()` returns ALL methods including bridge methods. Filter them with: `method.isBridge()` or `method.isSynthetic()`. If you iterate methods looking for your custom logic, exclude bridge methods. Bridge methods are not visible in source code but ARE present in bytecode — important for frameworks that use reflection (Spring AOP, Jackson) which must filter synthetics to avoid invoking the wrong method or getting confusing results.

Part 5 — Generics in Practice**Q22. How do you implement a type-safe heterogeneous container?**

Use `Class` as the key (type token): `put(Class<T> type, T value)` stores `type->value` in a `Map<Class<?>, Object>`. `get(Class<T> type)` retrieves and casts using `type.cast(map.get(type))` — which is a checked cast (throws `ClassCastException` safely if misused rather than silently corrupting). Limitation: can't distinguish `List` from `List` (both map to `List.class`). For parameterized types, use Guava `TypeToken` or Jackson `TypeReference`.

Q23. What is the super-type token pattern?

A technique to capture generic type information at runtime despite erasure. Create an anonymous subclass: `new TypeRef<>(){} — the anonymous class's superclass is TypeRef<>, a ParameterizedType. getClass().getGenericSuperclass() returns this ParameterizedType; ((ParameterizedType)pt).getActualTypeArguments()[0] returns List's Type object. Jackson's ObjectMapper uses this: mapper.readValue(json, new TypeReference<>()); to deserialize into a properly typed List<User>.`

Q24. What is the generic builder pattern and why use it?

A fluent builder where method chaining returns the correct subtype without casting. Uses F-bounded quantification: class `Builder<B extends Builder<B>>`. `self()` method returns `(B)this` — unchecked, but safe since `B` must be the actual subclass. Without generics, chaining methods in a superclass builder returns the supertype, forcing clients to cast: `(SubBuilder)parent.setName("x").setSubField("y")`. The generic builder eliminates those casts and maintains type safety through the chain.

Q25. How do you create a generic event bus?

```
Map<Class<?>, List<Consumer<?>>> handlers;
subscribe(Class eventType, Consumer handler):
handlers.computeIfAbsent(eventType, k->new CopyOnWriteArrayList<>()).add(handler);
publish(T event):
handlers.getOrDefault(event.getClass(), List.of()).forEach(h -> ((Consumer<T>)h).accept(event));
```

The unchecked cast (Consumer) is safe because subscribe enforces the type contract. Thread-safe with CopyOnWriteArrayList for handlers and ConcurrentHashMap for the map.

Advanced Questions — Senior/Principal Level

These questions probe deep JVM knowledge, type theory, and real-world design patterns. They appear in principal/staff engineer interviews.

Part 6 — Advanced Type Theory & JVM

[ADV] Q26. Why are Java generics invariant while arrays are covariant, and what problems does this cause?

Arrays are covariant: `String[] IS-A Object[]`. Generics are invariant: `List` is NOT `List`. Array covariance was a Java 1.0 design mistake — it allows: `Object[] arr = new String[5]; arr[0] = 42;` which compiles but throws `ArrayStoreException` at runtime (JVM checks array element type on every store). Generics chose invariance with wildcards for safety — `List` is the covariant analog, but you can't write to it. Arrays pay a runtime check cost; generics do it at compile time with erasure at runtime. Covariant generics (like Kotlin's `out T`) require read-only 'producer' semantics.

[ADV] Q27. What exactly is erased for different bounds: `, , & Serializable>`?

`erases to Object`. `erases to Number` (the first/only bound). `& Serializable>`: `T` erases to `Comparable` (first bound). Where `Serializable` is referenced, the compiler inserts a cast to `Serializable`. For multiple bounds, only the first bound appears in the erased signature — the rest are enforced via casts inserted by the compiler at usage sites. This is why the class bound must come first in `:` it determines the erasure type.

[ADV] Q28. How does the compiler enforce PECS — is it purely syntactic or does the JVM enforce it?

Purely compile-time enforcement — the JVM has no knowledge of wildcards. At bytecode level, `List` is just `List`. The compiler tracks the wildcard's bound in its symbol table and rejects method calls that would violate it (e.g., calling `add(E)` on a captured wildcard). After compilation, all wildcard information is gone. This means you can bypass PECS at runtime via raw types or unchecked casts — the JVM won't stop you, but you risk `ClassCastException` later.

[ADV] Q29. What is wildcard capture and why does it require a helper method?

When the compiler sees a method call on a wildcard-typed expression, it 'captures' the wildcard into a fresh type variable (CAP#1). This capture is local to that expression — two uses of the same `List` variable get different capture variables and the compiler treats them as potentially different types. So: `list.set(0, list.get(0))` fails because `list.get(0)` returns CAP#1 and `list.set(0, ...)` expects CAP#2 — different captures. The helper method `void helper(List l) { l.set(0, l.get(0)); }` works because the single `T` unifies both usages.

[ADV] Q30. Explain the recursive generic pattern class `Enum>` — what does it enable?

This F-bounded quantification means: `E` is a subtype of `Enum` — every enum implicitly has this bound. It enables: `Enum`'s `compareTo(E other)` method is type-safe — you can only compare an enum with members of the same enum type, not with arbitrary enums. Without this: `compareTo` would accept any `Enum`. With `E extends Enum`: `MyEnum.MON.compareTo(OtherEnum.X)` is a compile error. The pattern generalizes to fluent APIs where a method returns 'this' type: `method()` returns `B` (the actual subtype) rather than `Base` (the supertype).

[ADV] Q31. How does the `@SafeVarargs` annotation work and when is it appropriate?

`@SafeVarargs` on a final/static/private generic varargs method suppresses the 'possible heap pollution' warning. A `varargs T...` parameter creates a `T[]` — but arrays of non-reifiable types cause compiler warnings. When is it safe? Only when the method does NOT store into the varargs array and does NOT expose a reference to it. Example: `Arrays.asList(T... a)` is `@SafeVarargs` — it only reads from `a[]`. When is it NOT safe: if the method stores elements from a different parameterized type into the array, callers get unexpected `ClassCastExceptions`. `@SafeVarargs` is a promise to the compiler that you've verified the method is safe — it doesn't actually prevent unsafe code.

[ADV] Q32. How do generic type parameters interact with Java's module system (JPMS)?

Type parameters themselves are unaffected by modules — generics work the same within and across modules. However, reflection on generic types is affected: `getGenericType()` on a field in an unexported package will still return the `ParameterizedType`, but the actual type argument's `Class` object may be inaccessible if it's from an unexported package. Frameworks using `TypeReference`/`TypeToken` (Jackson, Guava) may fail with `InaccessibleObjectException` when trying to introspect generic supertype parameters of classes across module boundaries. Fix: open or export the relevant packages in `module-info.java`.

[ADV] Q33. What is the difference between `Class.cast()` and an explicit (T) cast?

`Class.cast(obj)` is a checked cast: it calls `Class.isInstance(obj)` first and throws `ClassCastException` immediately with a clear message if the types don't match. `(T) obj` is an unchecked cast — at bytecode level it's either a `checkcast` instruction (if `T` is known) or completely removed (if `T` is erased to `Object`). With erasure: `(T) obj` where `T` erases to `Object` generates NO `checkcast` — the cast silently succeeds and the `ClassCastException` occurs only when you try to use the value as `T`. `Class.cast()` is safer because it fails immediately at the cast site, not later.

[ADV] Q34. Why can't you instantiate a generic type with `new T()` and what are the design alternatives?

`new T()` is forbidden because: (1) after erasure `T` is `Object`; (2) `T` might not have a no-arg constructor; (3) `T` might be an interface or abstract class. Alternatives: (1) Pass `Class` token: `Class type; T obj = type.getDeclaredConstructor().newInstance();` (2) Pass `Supplier` factory: `Supplier factory; T obj = factory.get();` — preferred (no reflection). (3) Pass a prototype and clone it. `Supplier` is the modern idiom: `ThreadLocal.withInitial(ArrayList::new)` creates a new list per thread.

[ADV] Q35. How does Jackson's `TypeReference` work under the hood?

Jackson's `TypeReference` uses the super-type token pattern. When you write `new TypeReference(){} you create an anonymous subclass of TypeReference. The anonymous class's generic supertype is TypeReference — a ParameterizedType. TypeReference's constructor calls getClass().getGenericSuperclass() to get this ParameterizedType, then ((ParameterizedType)pt).getActualTypeArguments()[0] returns the Type for List<User>. This Type object is stored and later used by ObjectMapper to determine element type (User.class) and construct the appropriate deserializer. The JVM bytecode DOES retain generic signature metadata for class hierarchies — just not for local variables.`

[ADV] Q36. What is co-variance, contra-variance, and invariance in the context of generics?

Invariance: `List IS-NOT-A List` — Java generics default. Covariance: `List` — a list of some `Animal` subtype; read-only; analogous to Kotlin's `out T` or Scala's `+T`. Contravariance: `List` — a list of some `Animal` supertype; write-only for `Dog`; analogous to Kotlin's `in T` or Scala's `-T`. Java uses use-site variance (wildcards at each use), unlike Kotlin/Scala which support declaration-site variance (variance declared on the class itself). Declaration-site is cleaner but requires more design upfront.

[ADV] Q37. What is the 'get and put principle' and how does it guide wildcard usage?

The get-and-put principle (from Java Generics and Collections book): Use when you only GET values out of a structure (producer). Use when you only PUT values into a structure (consumer). Don't use wildcards when you both get and put. Example: sorting a list requires both getting (comparing) and putting (swapping) — use `List` with `>`, not a wildcard. This principle is exactly PECS but described from the perspective of the structure (what you do with it) rather than the data (what the structure does with the data).

[ADV] Q38. How does Guava `TypeToken` differ from Java's `TypeReference`, and when would you use each?

Java's `TypeReference` (Jackson/custom): captures one `ParameterizedType` via anonymous subclass. Simple to use for deserializing into one specific generic type. Guava `TypeToken`: a full type system over Java's `Type` hierarchy; supports type operations: `TypeToken.of(type).resolveType(T)`, `getSubtype(subclass)`, `getSupertype(superclass)`. Useful when you need to reason about generic types programmatically — e.g., checking if `TypeToken<List<String>>` is a subtype of `TypeToken<Collection<String>>`. Use `TypeReference` for simple serialization; use Guava `TypeToken` for framework-level generic type reasoning (e.g., building a DI container or ORM).

[ADV] Q39. What is Kotlin's reified type parameter and how does it solve Java's erasure problem?

Kotlin's inline functions with reified type parameters embed type information at call sites. `inline fun isType(obj: Any) = obj is T` — this compiles to a real instanceof check because the compiler substitutes the actual type T at each call site (since the function is inlined). Java cannot do this without passing Class because non-inline methods go through a single compiled bytecode that must work for all T. Kotlin's inline functions have separate bytecode per call site, allowing T to be a real runtime type. Java workaround: always pass Class as a parameter for operations that need runtime type.

[ADV] Q40. Explain how variance annotations work in Kotlin vs Java wildcards — a comparison.

Java: use-site variance — caller specifies variance at each usage: `List, List`. More flexible but verbose; every usage site must specify variance. Kotlin: declaration-site variance — the class declares variance once: `interface Source { fun nextT(): T }` (covariant/producer); `interface Sink { fun put(e: T) }` (contravariant/consumer). Callers get variance for free: `fun copy(from: Source, to: Sink)`. Declaration-site is cleaner for well-designed APIs. Java uses both implicitly: `Iterator` is conceptually covariant (only produces E); `Comparator` is contravariant (only consumes T for comparison) — but Java doesn't annotate these declarations.

Exercises

Part A — Coding Challenges

- A1.** Implement a generic Pair class with: (a) a static factory method of(A,B), (b) a swap() method returning Pair, (c) map(Function) returning Pair, (d) correct equals/hashCode/toString. Make it immutable with final fields.
- A2.** Write a generic method > List merge(List a, List b) that merges two sorted lists into one sorted list. Use PECS correctly — explain why the wildcards are needed.
- A3.** Implement a generic Result sealed type with Success and Failure records. Include: map(Function), flatMap(Function>), recover(Function), getOrThrow(). Demonstrate usage with a chain of fallible operations.
- A4.** Build a type-safe event bus: subscribe(Class eventType, Consumer handler) and publish(T event). Thread-safe using ConcurrentHashMap + CopyOnWriteArrayList. Handle the unchecked cast in publish() safely with a comment explaining why it's correct.
- A5.** Implement a generic memoize function: Function memoize(Function fn) that caches results in a HashMap. Then make a thread-safe version using ConcurrentHashMap and computeIfAbsent. Discuss the double-computation race condition and whether it matters.

Part B — Debug & Fix

B1 – Raw Type Heap Pollution

```
List rawList = new ArrayList<String>();
rawList.add(42); // no error!
List<String> strings = rawList;
String s = strings.get(0); // ClassCastException HERE (not at add!)
```

Fix: Adding to a raw type bypasses generic checks. The pollution is silent until you read. Fix: List list = new ArrayList<>(); — always specify type parameter. Enable -Xlint:unchecked to surface raw type warnings at compile time.

B2 – Cannot Overload by Generic Type

```
void process(List<String> list) { System.out.println("string"); }
void process(List<Integer> list) { System.out.println("int"); }
// COMPILE ERROR: both erase to process(List)
```

Fix: Both methods erase to process(List list) — identical signatures after erasure. Fix: use different method names (processStrings/processIntegers) or one method with Class parameter.

B3 – PECS Violation

```
void copyNumbers(List<? extends Number> src, List<? extends Number> dest) {
    for (Number n : src) dest.add(n); // COMPILE ERROR!
}
```

Fix: dest is — unknown subtype — you cannot add to it. Fix: make dest a consumer: List dest. Or use exact type: void copy(List src, List dest).

B4 – Generic Array Creation

```
public class MyStack<T> {
    private T[] elements = new T[16]; // COMPILE ERROR
}
```

Fix: Cannot create a generic array. Fix option 1: @SuppressWarnings("unchecked") T[] elements = (T[]) new Object[16]; Fix option 2(preferred): private List elements = new ArrayList<>(16); — avoids the issue entirely.

Part C — MCQ / Predict the Output

MC1. What is the output? List ls = new ArrayList<>(); List lw = ls; lw.add(null); System.out.println(lw.size());

- A) Compile error on add(null)
- B) 0
- C) 1
- D) NullPointerException

Answer: C — null is always allowed in a List even though typed values are forbidden. size() returns 1.

MC2. Which compiles?

- A) List li = new ArrayList();
- B) List ln = new ArrayList();
- C) List len = new ArrayList();
- D) List lsi = new ArrayList();

Answer: C — Generics are invariant — only C is correct. ? extends Number accepts ArrayList. D fails: String is not a supertype of Integer.

MC3. What does & Serializable> erase T to?

- A) Object
- B) Comparable
- C) Serializable
- D) Comparable & Serializable

Answer: B — T erases to its first bound: Comparable. Uses of Serializable methods get a cast inserted by the compiler.

MC4. Can you do instanceof with a wildcard? if (obj instanceof List) {}

- A) Yes — compiles fine
- B) Compile error — can only use instanceof with exact types
- C) Runtime warning
- D) Depends on Java version

Answer: A — instanceof with unbounded wildcard List is legal and reifiable. instanceof List is illegal (non-reifiable). List is reifiable.

MC5. What does method.isBridge() return for a compiler-generated bridge method?

- A) false
- B) true
- C) throws UnsupportedOperationException
- D) It depends on the JVM

Answer: B — Bridge methods (generated by compiler for erasure + polymorphism) have ACC_BRIDGE flag in bytecode. isBridge() returns true for them; isSynthetic() also returns true.

MC6. Which is the correct way to get a generic type's runtime info at runtime?

- A) T.class
- B) getClass() on an instance of T
- C) Subclass TypeRef and call getClass().getGenericSuperclass()
- D) Not possible — erasure makes it impossible

Answer: C — The super-type token pattern captures ParameterizedType in the class hierarchy metadata. T.class is a compile error; getClass() returns the concrete class without type params.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Generic class	class Foo{}; diamond operator <>; raw types bypass type safety
Type param names	T=Type, E=Element, K=Key, V=Value, N=Number, R=Return
Bounded param	: T must be Number/subtype; first bound = erasure type
Multiple bounds	— class bound MUST be first
F-bounded	>: T refers to itself; enables type-safe compareTo
Wildcard	Unknown type; read as Object; cannot add (only null); max flexibility
PECS	Producer Extends (read), Consumer Super (write); exact type for both
	Read returns T; add blocked; covariant; use for read-only sources
	Read returns Object; add T allowed; contravariant; use for write-only sinks
Type erasure	Compile-time only; erased to bound (or Object) at runtime; backward compat
Erasure limits	No new T(); no T[]; no instanceof List; no catch generic; no overload by generic
Heap pollution	Mixing raw+generic allows wrong types silently; ClassCastException elsewhere
@SafeVarargs	On final/static/private generic varargs: promise not to store/expose the array
Bridge method	Compiler-generated for erasure+polymorphism; isBridge()+isSynthetic()==true
Type token	Class for runtime type; super-type token for ParameterizedType (TypeRef pattern)
Generic array	Avoid; use (T[])new Object[n] with @SuppressWarnings or List
Variance	Java: use-site (wildcards); Kotlin: declaration-site (out/in); invariant by default
Wildcard capture	Compiler captures ; helper method unifies captures for type-safe ops

End of Sub-Chapter 1.7 — Generics

Next: Sub-Chapter 1.8 — Exception Handling